

多模态时间序列预测：对于文本模态融合的有效性与机制初步探究

姓名：吴晨曦

学号：10242150443

项目代码仓库：<https://github.com/wcxxx57/AI-fundamental-TSF>

1. 实验任务与探究问题

1.1 任务定义

时间序列预测任务基于历史信息来预测变量未来的值。给定长度为 H 的历史序列 X ，预测长度为 F 的未来序列 Y ：

$$X_{t-H+1:t} \rightarrow Y_{t+1:t+F}$$

本项目聚焦于多模态时序预测，重点探究融合了文本模态的时间序列预测是否会更加有效，以及怎样的文本和融合机制更可能带来稳定收益。

1.2 核心探究问题

本项目主要围绕以下三个问题展开两个对比实验。前两个问题对应对比实验一，最后一个问题对应对比实验二：

1. 补充文本模态是否有效？

基于强数值 baseline (DLinear 和 PatchTST)，探究额外的文本模态是否真的能带来稳定提升。

2. 怎样的文本模态有效？

比较三种来源和性质不同的文本模态：随机文本、LLM 根据历史时间序列自生成的描述文本、Time-MMD 外源 news/report 文本，分别会带来怎样不同的效果。

3. 如何使文本模态有效？

比较三种 prediction-level 融合机制：MM-TSFlib 基础线性融合、自适应非线性融合、频域分解融合，观察不同融合机制是否能更好地释放文本分支的价值。

2. 数据集与实验设置

2.1 Time-MMD 数据集

Time-MMD 是一个多领域多模态时间序列数据集，数值序列来自政府、行业 and 公共数据源，文本序列由**报告**（Report）与**网络搜索**（Web Search）两种渠道聚合而成。本项目选取其中 8 个领域，统一使用数据文件中的 `Final_Search_4` 列作为外源文本输入。

领域	数据文件	频率	目标变量	变量含义	外源文本列
Agriculture	<code>US_RetailBroilerComposite_Month.csv</code>	月度	<code>OT</code>	零售肉鸡综合价格	<code>Final_Search_4</code>
Climate	<code>US_precipitation_month.csv</code>	月度	<code>OT</code>	美国月度降水量	<code>Final_Search_4</code>
Economy	<code>US_TradeBalance_Month.csv</code>	月度	<code>OT</code>	美国贸易差额	<code>Final_Search_4</code>
Energy	<code>US_GasolinePrice_Week.csv</code>	周度	<code>OT</code>	美国汽油价格	<code>Final_Search_4</code>
Public_Health	<code>US_FLURATIO_Week.csv</code>	周度	<code>OT</code>	流感比例	<code>Final_Search_4</code>
Security	<code>US_FEMAGrant_Month.csv</code>	月度	<code>OT</code>	FEMA 灾害救助拨款	<code>Final_Search_4</code>
SocialGood	<code>Unadj_UnemploymentRate_ALL_processed.csv</code>	月度	<code>OT</code>	未经季调失业率	<code>Final_Search_4</code>
Traffic	<code>US_VMT_Month.csv</code>	月度	<code>OT</code>	车辆行驶里程	<code>Final_Search_4</code>

2.2 随机文本与LLM自生文本数据构建

为了探究"怎样的文本模态有效"的问题，我们在 Time-MMD 原始外源文本之外，额外构建了随机文本和LLM自生成文本，形成纯数值与三种文本模态的对比：

文本类型	实验名	CSV 文本列	构建方式
纯数值 (无文本)	<code>numeric</code>	无	只训练 DLinear/PatchTST
随机文本	<code>random_text</code>	<code>Random_Text</code>	字符级随机噪声，长度与外源文本近似
LLM 自生成 文本	<code>llm_generated</code>	<code>ECNU_LLM_Text</code>	ECNU LLM 只根据历史窗口生成事实描述和定性预测
外源文本	<code>mm_tsflib</code>	<code>Final_Search_4</code>	Time-MMD 提供的 news/report/search 文本

这四组实验共享同一套数据切分、backbone、训练协议和融合路径。对于三种文本输入，模型结构完全一致，唯一的区别是读取的文本列不同。这种设计让任何性能差异都可以更好地"控制变量"，干净地归因到"文本内容"而非"模型结构"。

随机文本控制组

随机文本由 `scripts/build_random_text_control_dataset.py` 生成。核心设计是：只从 26 个英文字母、空格和少量标点中采样，生成看起来有英文形态但完全没有语义的信息。长度默认与同一行 `Final_Search_4` 的字符数匹配，避免"文本长度"成为额外变量。

```
LETTERS = "abcdefghijklmnopqrstuvwxyz"

def random_char_sentence_text(target_chars: int, rng: random.Random) -> str:
    parts = []
    current_len = 0
    while current_len < target_chars:
        word_len = rng.randint(2, 9)
        word = "".join(rng.choice(LETTERS) for _ in range(word_len))
        if not parts:
            word = word.capitalize()
        if rng.random() < 0.08:
            word += rng.choice(["", ".", "!", "?", ";"])
        candidate = word if not parts else " " + word
        parts.append(candidate)
        current_len += len(candidate)
    return "".join(parts).strip()
```

每一行随机文本都用 `row_index` 和固定 `seed` 做确定性生成，保证可复现。同时写入 audit JSONL，明确标记：

```
{
  "uses_target_values": false,
  "uses_future_rows": false,
  "rule": "Text is generated from deterministic random letters, spaces, and
punctuation; source columns are used only for length control."
}
```

下面是 Climate 数据集前 3 行随机文本示例（按 `seed=20240624` 复现，实际文件保留完整长度）：

row	date	OT	Random_Text 示例（前 120 字符）
23	1984-12-01	2.49	Vxvwvk ebdubxun ozgkqedr xaphn psoudzpq whlxnrqgq cio qri axbn da? Nxyomqx lu ryegbhliz...
24	1985-01-01	1.42	Lpcqnrjw zcohgb yeca bo tqrxdnruw ws pjwklr moeee! Yikg imnqlei nsf mf anfeqfwrdeq...
25	1985-02-01	2.10	Hw dgbgrljt fjiiawc qqeyze jl vca gy jnjmn. Yhngqwrrp ikryxee xet, cyxup ckzfi...

LLM 自生成文本

LLM 自生成文本由 `scripts/build_ecnu_llm_text_dataset.py` 构建，具体 prompt 模板可在其中查看。总体构建生成目标不是让 LLM 预测未来数值，而是将数值序列信息转换成相应的文本模态而不添加额外的外部信息。为了和TIME-MMD数据集中的文本结构类似，构建时把历史窗口中的数值变化转换成两段短文本：

- `fact`：只描述历史窗口内已经观察到的事实，例如趋势、波动、峰谷、转折点；
- `pred`：基于同一历史窗口做定性预测分析，例如可能延续、反转或存在不确定性，但禁止输出精确未来值。

下面是 Climate 数据集中 2 行最终生成的 `ECNU_LLM_Text` 示例：

date	OT	ECNU_LLM_Text
1984-12-01	2.49	Fact: US monthly precipitation fluctuated between 1.35 and 3.91 from 1983-1984. A sharp low occurred in Jan 1984. The series shows recurring seasonal patterns with peaks in late spring/autumn and troughs in mid-summer, ending at 2.49 in Dec 1984. Pred: Given the seasonal cycle, early 1985 may see lower winter precipitation followed by a spring rise. Volatility suggests uncertainty in exact levels. No strong trend implies continuation of historical oscillation rather than a directional shift in wetness.
1985-01-01	1.42	Fact: US precipitation from Feb 1983 to Jan 1985 fluctuates between 1.35 and 3.91. Two distinct peaks occur in March 1983 and November 1983. A pronounced dry spell appears in early 1984, followed by moderate levels. The most recent observation in January 1985 is 1.42, a local low. Pred: Given the seasonal pattern with winter lows and spring/autumn peaks, precipitation may rise moderately in early 1985. However, high volatility exists. The recent low suggests a potential wetting trend, but historical consistency is limited. Expect fluctuations around the multi-year mean without extreme certainty.

泄漏检查与控制。 代码层面做了三件事来保证时间对齐：

1. 数据加载时设置 `text_origin_offset = -1`，即文本行索引为 `s_end - 1`，对应历史窗口的最后一个时间点（origin），而不是未来时刻。
2. `prior_mode = origin_repeat`，即 `prior` 分支只把 `origin` 时刻已知的 `prior_history_avg` 重复到未来窗口，不读取未来真实值。
3. LLM 生成数据保存 `audit` 字段，记录 `max_prompt_row_index <= origin_row_index`，并明确声明 `prompt` 中只包含历史序列和变量含义，不包含任何未来信息。

```
{
  "uses_y_values": false,
  "max_prompt_row_index": 23,
  "origin_row_index": 23,
  "max_prompt_time": "1984-12-01",
  "origin_time": "1984-12-01",
  "uses_external_or_future_information": false,
  "rule": "max_prompt_row_index <= origin_row_index; prompt contains only
serialized historical rows and user-supplied variable meanings/domain context"
}
```

2.3 预测 setting 与训练协议

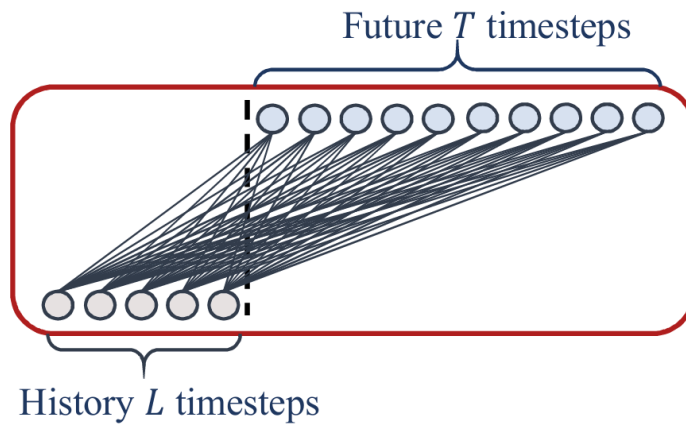
参数	取值	说明
seq_len	24	历史窗口长度。月度数据约 2 年，周度数据约半年
label_len	12	decoder 已知标签长度
pred_len	12, 24, 36, 48	多 horizon 预测。Economy 只保留 12/24/36, Security 只保留 12/24
features	S	单变量预测单变量
target	OT	每个数据集的目标列
切分方式	7:2:1	按时间顺序切分，训练/验证/测试不随机打乱
归一化	StandardScaler	只在训练集 fit，验证/测试只做 transform
指标	MSE, MAE	主报告使用标准化尺度 test MSE/MAE
backbone	DLinear, PatchTST	一个线性强基线，一个 Transformer-based 强基线
seed	2021	当前报告为单 seed 结果
epoch/patience	10 / 5	矩阵默认 10 epochs, early stopping patience=5
batch size	32	所有主矩阵一致
prompt weight	0.1	基础融合中 $Y_{final} = 0.9 * Y_{num} + 0.1 * Y_{text}$

3. 核心模型与融合设计

3.1 数值基线模型

DLinear

DLinear 是本项目的使用的轻量线性 baseline。它先把序列通过移动平均分解为 seasonal 和 trend 分量，再分别用单个线性层把 `seq_len` 映射到 `pred_len`，如下图所示：

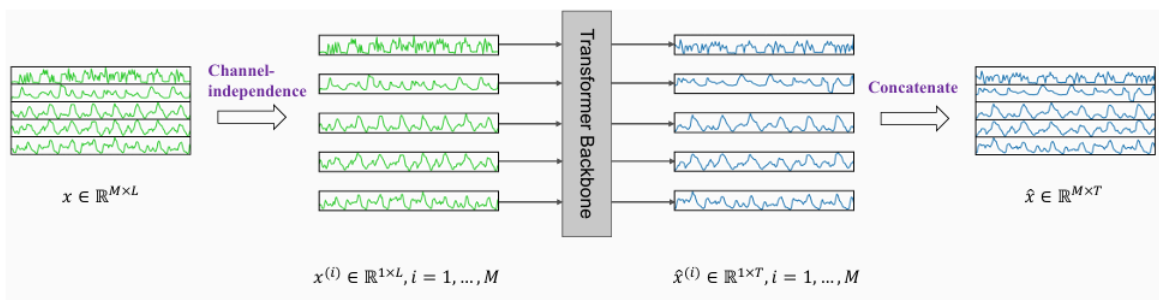


```
class DLinear(nn.Module):
    def __init__(self, configs, individual=False):
        self.decompstion = series_decomp(configs.moving_avg)
        self.Linear_Seasonal = nn.Linear(self.seq_len, self.pred_len)
        self.Linear_Trend = nn.Linear(self.seq_len, self.pred_len)

    def encoder(self, x):
        seasonal_init, trend_init = self.decompstion(x)
        seasonal_output = self.Linear_Seasonal(seasonal_init)
        trend_output = self.Linear_Trend(trend_init)
        return (seasonal_output + trend_output).permute(0, 2, 1)
```

PatchTST

PatchTST 是 Transformer-based 的数值基线模型。它把时间序列切成 patch，再用 Transformer Encoder 建模 patch 间关系，通常比简单线性模型更强，模型基本架构如下图所示：



```

enc_out, n_vars = self.patch_embedding(x_enc)
enc_out = self.encoder(enc_out)
enc_out = self.head(enc_out)
forecast = enc_out.permute(0, 2, 1) * stdev + means

```

3.2 MM-TSFLib 基础线性融合机制

基础多模态模型继承 MM-TSFLib 的 prediction-level fusion 思路。

- 数值分支：由数值 backbone（本项目使用DLinear和PatchTST）生成 `num_pred`
- 文本分支：经过 tokenizer → BERT embedding → MLP → pooling 生成 `text_signal`
文本信号，再与 `origin` 时刻的历史均值先验 `prior_y` 相加得到 `prompt_y`

最后将数值分支和文本分支按固定权重融合：

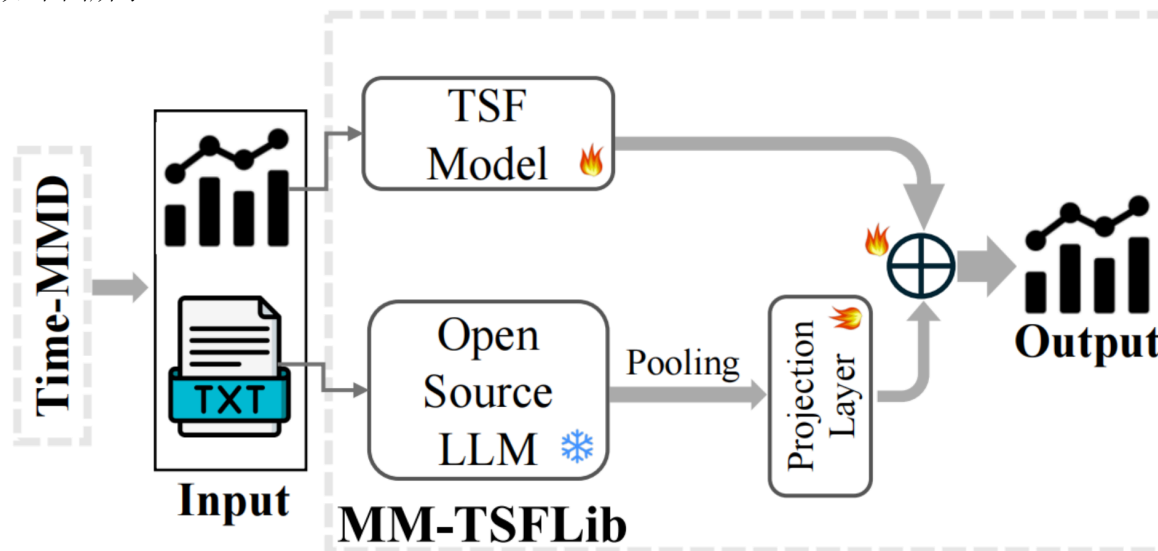
```

历史数值 X -> DLinear/PatchTST -> num_pred
文本 S -> tokenizer -> BERT embedding -> MLP -> pooling -> text_signal
origin prior -> prior_y

prompt_y = norm(text_signal) + prior_y
Y_final = (1 - prompt_weight) * num_pred + prompt_weight * prompt_y

```

如下图所示：



核心代码（文本编码与融合部分）：

```

# 1. 取 origin 时刻的历史先验和对应文本
prior_y = torch.from_numpy(train_data.get_prior_y(index)).float().to(self.device)
batch_text = train_data.get_text(index)

# 2. 拼接 prompt 并获取 BERT input embeddings
prompt = [

```



```

        f"<|start_prompt|Make predictions...{text_info}<|<end_prompt>|>"
        for text_info in batch_text
    ]
    token_ids = self.tokenizer(prompt, return_tensors="pt", padding=True,
                               truncation=True, max_length=1024).input_ids
    prompt_embeddings = self.llm_model.get_input_embeddings()(
        token_ids.to(self.device)
    )

    # 3. MLP 投影 + average pooling
    prompt_emb = self.mlp(prompt_emb)
    prompt_emb = F.adaptive_avg_pool1d(
        prompt_emb.transpose(1, 2), 1
    ).squeeze(2).unsqueeze(-1)

    # 4. norm 后与 prior 相加，再与数值预测融合
    prompt_y = norm(prompt_emb) + prior_y
    outputs = (1 - self.prompt_weight) * outputs + self.prompt_weight * prompt_y

```

这里有两个需要注意的细节：

- 当前矩阵中 `use_fullmodel=0`，文本编码没有跑完整 BERT contextual forward，而是使用 BERT input embedding + MLP + pooling。这样降低了计算量，但也会削弱模型对真实语义差异的敏感性。
- 实验采用的 `prompt_weight=0.1`，即最终预测 90% 来自数值 backbone，10% 来自文本/prior 分支（这个 `prompt_weight` 其实是就超参数，我在问辩的时候和杨老师解释的时候没搞清楚说错了）。这个 `prompt_weight=0.1` 的实验设置也使后面实验 LLM、external、random 三种文本指标相对比较接近，不过依然能够看出其彼此的大小。

三种文本实验的实现差异仅在文本来源和性质方面：继承同一基类，只改变默认文本列和 prior 模式。这保证了对比实验一的控制变量清楚。

```

class Exp_Random_Text_Forecast(Exp_Long_Term_Forecast):
    def __init__(self, args):
        args.text_column = "Random_Text"
        args.text_origin_offset = -1
        args.prior_mode = "origin_repeat"
        super().__init__(args)

class Exp_LLM_Generated_Forecast(Exp_Long_Term_Forecast):
    def __init__(self, args):
        args.text_column = "ECNU_LLM_Text"
        args.text_origin_offset = -1
        args.prior_mode = "origin_repeat"
        super().__init__(args)

```

3.3 自适应非线性融合

设计动机

基础线性融合使用全局固定的 `prompt_weight=0.1`，这意味着每个样本、每个时刻点都按同样的 9:1 比例混合数值预测和文本/prior 预测。但直觉上，不同样本的"文本可信度"可能不同：有的样本历史波动剧烈、文本信息丰富，可能应该更依赖文本；有的样本本身趋势稳定、文本又是噪声，就应该更信任数值 backbone。非线性融合的目标是学习一个样本级的自适应门控，让每个样本动态决定数值与文本的融合比例。

融合机制

非线性融合在 prediction level 学习一个 gate。对于每个时间步的三个标量特征——数值预测 Y_{num} 、文本/prior 预测 Y_{text} 、以及它们的差值 $Y_{text} - Y_{num}$ （表示文本分支建议的"修正方向"）——拼接后送入一个轻量 MLP，输出 sigmoid gate 值 $G \in (0, 1)$ ：

$$Z = [Y_{num}, Y_{text}, Y_{text} - Y_{num}]$$

$$G = \sigma(\text{MLP}(Z)), \quad G \in (0, 1)$$

$$Y = Y_{num} + G \cdot (Y_{text} - Y_{num})$$

当 $G \approx 0$ 时，输出几乎等于数值预测；当 $G \approx 1$ 时，输出几乎等于文本预测。这样每个样本、每个时间步都可以独立学习融合权重。

保守初始化策略

非线性融合有一个关键风险：如果 gate 初始化不当，训练初期可能大幅偏离数值 backbone 的已有预测，导致训练不稳定甚至崩溃。为此，我们采用**保守初始化**：将 gate MLP 的最后一层权重置零，bias 设为 `prompt_weight` 对应的 logit 值：

$$b_{gate} = \log\left(\frac{w}{1-w}\right), \quad w = \text{prompt_weight}$$

这样训练开始时 $G \approx w$ ，整个融合模块等价于基础线性融合。后续训练只在"需要偏离"时才逐渐调整 gate 值。这种"从已知有效配置出发"的初始化策略，避免了非线性融合一上来就破坏数值 backbone 的预测。

核心代码如下：

```
class SimpleAdaptiveGateFusion(nn.Module):
```

```
"""自适应门控融合：每个样本动态学习数值与文本的融合权重。"""
```

```
def __init__(self, init_weight=0.01, hidden_dim=8):
    super().__init__()
    self.gate = nn.Sequential(
        nn.Linear(3, hidden_dim),
        nn.GELU(),
        nn.Linear(hidden_dim, 1),
    )
    self._reset_to_linear_start()

def _reset_to_linear_start(self):
    """保守初始化：训练开始时等价于基础线性融合。"""
    clipped = min(max(self.init_weight, 1e-4), 1.0 - 1e-4)
    gate_bias = float(np.log(clipped / (1.0 - clipped)))
    nn.init.zeros_(self.gate[-1].weight)
    nn.init.constant_(self.gate[-1].bias, gate_bias)

def forward(self, num_pred, prompt_y):
    # 拼接 [Y_num, Y_text, Y_text - Y_num]
    features = torch.cat(
        [num_pred, prompt_y, prompt_y - num_pred], dim=-1
    )
    gate = torch.sigmoid(self.gate(features))
    return num_pred + gate * (prompt_y - num_pred)
```

3.4 频域分解融合

设计动机

频域融合受 VoT (VALUE OF TEXT) 思想的启发：文本信息可能更适合影响序列的**低频趋势成分**（如长期走向、季节性漂移），而不一定适合解释**高频噪声成分**（如短期随机波动）。如果强行在时域做全局融合，文本可能会“污染”数值 backbone 已经很好建模的高频部分。频域融合的思路是：将数值预测和文本/prior 预测分别分解到不同频率段，然后在每个频段上独立学习融合权重。

分解与融合流程

1. **频域分解**：对数值预测 `num_pred` 和文本/prior 预测 `prompt_y` 分别做一维实数 FFT（`rfft`），将时域信号拆成三个频带：
 - **低频分量**：序列的整体趋势和长期模式（频率索引 `[0, low_cutoff)`）
 - **中频分量**：中等周期的波动模式（频率索引 `[low_cutoff, high_cutoff)`）
 - **高频分量**：短期噪声和快速变化（频率索引 `[high_cutoff, freq_len)`）频带边界由超参数 `low_freq_ratio=0.1` 和 `high_freq_ratio=0.3` 控制。
2. **跨模态加权融合**：六个分量（数值低/中/高 + 文本低/中/高）堆叠后，用六个可学习标量权重 `decomp_w` 加权求和：

$$Y_{fused} = w_1Y_{num}^{low} + w_2Y_{num}^{mid} + w_3Y_{num}^{high} + w_4Y_{text}^{low} + w_5Y_{text}^{mid} + w_6Y_{text}^{high}$$

3. 时域恢复：融合后的频域信号通过 `irfft` 逆变换回时域，得到最终预测。

保守初始化策略

与非线性融合类似，频域融合也需要保守初始化。六个频段的初始权重按 `1 - prompt_weight` 和 `prompt_weight` 分配：

```
init_weights = [0.9, 0.9, 0.9, 0.1, 0.1, 0.1]
```

这保证训练开始时，每个频段都近似做基础线性融合（90% 数值 + 10% 文本），后续只在特定频段上学习偏离。

三种融合机制对比

特性	基础线性融合	自适应非线性融合	频域分解融合
融合粒度	全局固定权重	样本级动态权重	频段级动态权重
可学习参数	无（ <code>prompt_weight</code> 固定）	MLP gate（约 40 参数）	6 个频域权重
初始化策略	—	等价于线性融合起点	每个频段等价于线性融合
核心假设	所有样本文本贡献相同	不同样本文本可信度不同	文本更适合影响低频趋势
优点	简单稳定	轻量、退化风险低	可捕捉频域结构化差异
潜在风险	无法自适应	改善幅度可能有限	高频超参数敏感、可能退化

4. 实验运行与矩阵

4.1 实验入口

所有实验可通过 `run.py` 中的 `--experiment` 参数统一路由到对应的实验类：

```
if args.experiment == "numeric":
    Exp = Exp_Numeric_Long_Term
elif args.experiment == "mm_tsflib":
    Exp = Exp_Long_Term_Forecast
elif args.experiment == "mm_tsflib_nonlinear":
    Exp = Exp_Nonlinear_Fusion_Forecast
elif args.experiment == "mm_tsflib_vot_freq":
    Exp = Exp_VoT_Frequency_Fusion_Forecast
elif args.experiment == "llm_generated":
    Exp = Exp_LLM_Generated_Forecast
elif args.experiment == "random_text":
    Exp = Exp_Random_Text_Forecast
```

4.2 实验矩阵

数值backbone组合

本项目使用D Linear 和 PatchTST 两种数值backbone，共 2 种组合情况

数据集组合

本项目使用的 TIME-MMD 的 8 个数据集各自可用的预测步长不同，组合总数为 29：

数据集	pred_len 组合	组合数
Agriculture	12 / 24 / 36 / 48	4
Climate	12 / 24 / 36 / 48	4
Economy	12 / 24 / 36	3
Energy	12 / 24 / 36 / 48	4
Public_Health	12 / 24 / 36 / 48	4
Security	12 / 24	2
SocialGood	12 / 24 / 36 / 48	4
Traffic	12 / 24 / 36 / 48	4
合计	—	29

实验组合

- 文本模态对比（实验一）：4 种输入配置（`numeric`、`random_text`、`llm_generated`、`mm_tsflib`）
 - $4 \times 2 \text{ backbone} \times 29 = 232 \text{ run}$
- 融合机制对比（实验二）：3 种融合方式（`mm_tsflib`、`mm_tsflib_nonlinear`、`mm_tsflib_vot_freq`）
 - $3 \times 2 \text{ backbone} \times 29 = 174 \text{ run}$

去重与总 run 数

`mm_tsflib` 同时出现在两个实验中，去重后 unique setting 数为 $232 + 174 - 58 = 348$ 。矩阵实际跑了 `itr = 2` 轮（每轮独立训练与测试），因此顶层总 run 数为 $348 \times 2 = 696$ 。

类型	计算方式	run 数
文本模态对比	$4 \text{ exp} \times 2 \text{ backbone} \times 29$	232
融合机制对比	$3 \text{ exp} \times 2 \text{ backbone} \times 29$	174
去重后 unique setting	$232 + 174 - 58$ (<code>mm_tsflib</code> 重复)	348
顶层主矩阵总 run	$348 \times 2 \text{ itr}$	696
NaN / 无效 run	—	0

通过 `scripts/run_long_matrix.sh` 脚本可直接跑矩阵测试。

5. 对比实验一：文本模态是否有效，以及怎样的文本有效

5.1 实验设计

本实验固定融合机制为 MM-TSFlib 基础线性融合，在下对比四种输入配置：

```
Numeric（无文本）
Random Text（随机噪声）
LLM-generated Text（历史窗口自生成）
External Text（Time-MMD Final_Search_4）
```

5.2 结果表

每种配置都运行 DLinear 和 PatchTST 两个 backbone 。括号里的值是相对同 backbone Numeric baseline 的绝对变化，负数表示误差降低：

dataset	pred_lens	metric	DLinear Numeric	DLinear Random	DLinear LLM	DLinear External	PatchTST Numeric	PatchTST Random	PatchTST LLM	PatchTST External
Algriculture	12/24/36/48	MSE	1.450	1.418 (-0.032)	1.420 (-0.030)	1.417 (-0.033)	1.096	1.030 (-0.065)	1.031 (-0.065)	1.031 (-0.065)
Algriculture	12/24/36/48	MAE	0.833	0.819 (-0.013)	0.820 (-0.013)	0.819 (-0.013)	0.719	0.704 (-0.015)	0.704 (-0.015)	0.704 (-0.015)
Climate	12/24/36/48	MSE	1.187	1.112 (-0.075)	1.112 (-0.075)	1.112 (-0.075)	1.176	1.059 (-0.117)	1.057 (-0.119)	1.060 (-0.116)
Climate	12/24/36/48	MAE	0.862	0.835 (-0.027)	0.835 (-0.028)	0.835 (-0.027)	0.855	0.816 (-0.039)	0.816 (-0.039)	0.817 (-0.038)
Economy	12/24/36	MSE	0.555	0.536 (-0.019)	0.532 (-0.023)	0.536 (-0.019)	0.318	0.303 (-0.015)	0.304 (-0.014)	0.303 (-0.015)
Economy	12/24/36	MAE	0.612	0.604 (-0.008)	0.601 (-0.011)	0.604 (-0.008)	0.449	0.435 (-0.014)	0.436 (-0.014)	0.435 (-0.015)
Energy	12/24/36/48	MSE	0.347	0.318 (-0.029)	0.318 (-0.029)	0.318 (-0.029)	0.253	0.254 (+0.002)	0.255 (+0.002)	0.254 (+0.001)
Energy	12/24/36/48	MAE	0.453	0.418 (-0.035)	0.418 (-0.035)	0.418 (-0.035)	0.362	0.366 (+0.005)	0.367 (+0.005)	0.366 (+0.004)
Public_Health	12/24/36/48	MSE	2.441	1.934 (-0.507)	1.930 (-0.511)	1.934 (-0.507)	1.744	1.590 (-0.154)	1.589 (-0.155)	1.589 (-0.156)
Public_Health	12/24/36/48	MAE	1.148	0.961 (-0.187)	0.959 (-0.188)	0.961 (-0.187)	0.892	0.835 (-0.056)	0.837 (-0.055)	0.836 (-0.056)
Security	12/24	MSE	73.639	73.198 (-0.441)	73.195 (-0.444)	73.195 (-0.444)	83.160	81.877 (-1.283)	81.241 (-1.919)	81.804 (-1.356)
Security	12/24	MAE	4.237	4.137 (-0.100)	4.137 (-0.100)	4.137 (-0.100)	5.101	5.001 (-0.099)	4.964 (-0.137)	4.996 (-0.104)
SocialGood	12/24/36/48	MSE	1.761	1.527 (-0.234)	1.529 (-0.233)	1.528 (-0.234)	1.514	1.518 (+0.004)	1.531 (+0.017)	1.515 (+0.001)
SocialGood	12/24/36/48	MAE	1.004	0.931 (-0.073)	0.929 (-0.075)	0.931 (-0.073)	0.802	0.794 (-0.008)	0.790 (-0.012)	0.791 (-0.011)
Traffic	12/24/36/48	MSE	0.323	0.295 (-0.027)	0.295 (-0.028)	0.294 (-0.029)	0.296	0.283 (-0.013)	0.284 (-0.012)	0.283 (-0.013)
Traffic	12/24/36/48	MAE	0.439	0.428 (-0.011)	0.428 (-0.011)	0.429 (-0.010)	0.366	0.348 (-0.018)	0.349 (-0.017)	0.348 (-0.018)

从结果表可以看出（加粗划线的为最小MSE/MAE）：
“加上文本模态”具有明显的增益效果：DLinear 上三种文本在全部 8 个领域都降低了 MSE；PatchTST 上也有 6-7 个领域改善。
不过，由于PatchTST 本身已经很强，在 Energy 和 SocialGood 等数据集上，加文本反而轻微退化，这也说明了基本数值模型越强，其加上文本模态的效果呈现一定的边际效应。

另外还需要说明的是，从结果表中可以看出三种文本模式指标的后三位小数点差别都很微弱，DLinear 上很多 LLM-generated 和 external 的值看起来甚至都和 random 差不多。我认为其原因是最终预测中文本分支 `prompt_weight=0.1`，即90% 来自数值 backbone，且三种文本共享 `origin_repeat` prior；同时由于实验设置的 `use_fullmodel=0`，文本编码没有跑完整 BERT contextual forward，削弱了模型对真实语义差异的敏感性，最终导致文本差异被这个架构压得很小。不过依然可以从结果看出LLM-generated和external文本强于random的大小趋势。

6. 对比实验二：融合机制如何影响文本模态的效果

6.1 实验设计

本实验固定数据集为 Time-MMD 外源文本（`Final_Search_4`），固定 backbone，对比三种融合机制：

Linear Fusion: MM-TSFlib 基础线性融合（固定权重 0.1）
Nonlinear Fusion: 自适应 MLP gate（样本级动态权重）
Frequency Fusion: VoT-style 频域分解融合（频段级动态权重）

6.2 结果表

同样在 DLinear 和 PatchTST 两个 backbone 下比较。括号里的值是相对同 backbone Linear Fusion 的绝对变化，负数表示更好：

dataset	pred_lens	metric	DLinear	Linear	DLinear Nonlinear	DLinear Frequency	PatchTST Linear	PatchTST Nonlinear	PatchTST Frequency
Algriculture	12/24/36/48	MSE	1.417		1.417 (+0.000)	1.463 (+0.046)	1.031	1.031 (-0.000)	1.188 (+0.157)
Algriculture	12/24/36/48	MAE	0.819		0.819 (-0.000)	0.829 (+0.010)	0.704	0.702 (-0.002)	0.740 (+0.036)
Climate	12/24/36/48	MSE	1.112		1.106 (-0.006)	1.093 (-0.019)	1.060	1.058 (-0.002)	1.039 (-0.021)
Climate	12/24/36/48	MAE	0.835		0.833 (-0.002)	0.828 (-0.007)	0.817	0.817 (+0.000)	0.808 (-0.008)
Economy	12/24/36	MSE	0.536		0.530 (-0.006)	0.551 (+0.014)	0.303	0.296 (-0.007)	0.328 (+0.025)
Economy	12/24/36	MAE	0.604		0.601 (-0.003)	0.613 (-0.009)	0.435	0.430 (-0.005)	0.456 (+0.021)
Energy	12/24/36/48	MSE	0.318		0.317 (-0.001)	0.316 (-0.002)	0.254	0.253 (-0.001)	0.248 (-0.006)
Energy	12/24/36/48	MAE	0.418		0.416 (-0.001)	0.417 (-0.001)	0.366	0.362 (-0.005)	0.357 (-0.009)
Public_Health	12/24/36/48	MSE	1.934		1.904 (-0.030)	1.872 (-0.062)	1.589	1.532 (-0.057)	1.536 (-0.053)
Public_Health	12/24/36/48	MAE	0.961		0.953 (-0.008)	0.930 (-0.031)	0.836	0.822 (-0.013)	0.809 (-0.027)
Security	12/24	MSE	73.195		73.176 (-0.019)	73.205 (+0.010)	81.804	80.005 (-1.799)	81.240 (-0.564)
Security	12/24	MAE	4.137		4.135 (-0.002)	4.134 (-0.003)	4.996	4.845 (-0.152)	4.945 (-0.051)
SocialGood	12/24/36/48	MSE	1.528		1.515 (-0.013)	1.402 (-0.126)	1.515	1.505 (-0.010)	1.341 (-0.173)
SocialGood	12/24/36/48	MAE	0.931		0.925 (-0.006)	0.877 (-0.054)	0.791	0.786 (-0.005)	0.732 (-0.058)
Traffic	12/24/36/48	MSE	0.294		0.292 (-0.002)	0.295 (+0.002)	0.283	0.282 (-0.001)	0.235 (-0.049)
Traffic	12/24/36/48	MAE	0.429		0.422 (-0.007)	0.431 (+0.002)	0.348	0.346 (-0.002)	0.312 (-0.036)

从结果可以看出（加粗划线的为最小MSE/MAE）：

融合机制的改进对于文本数值的融合有明显的增益效果，其中：

- **非线性融合**的特点是更稳：MSE 在 DLinear 7/8、PatchTST 8/8 个领域上优于线性融合，但平均改善幅度不大（0.6%–1.3%）。它更像是对固定 `prompt_weight` 的轻量校准，不容易破坏数值 backbone 的预测。
- **频域融合**的特点是更激进：平均收益在 PatchTST 上更高（1.7%–1.9%），在 SocialGood、Traffic、Climate、Public_Health 上改善明显，但在 Agriculture 和 Economy 上明显退化。这说明"按频段学习文本残差"有潜力，但对数据集特性更敏感。

7.讨论及结论

7.1 对三个研究问题的回答

问题一：补充文本模态是否有效？

在当前单 seed、当前数据切分和当前实现下，补充文本模态相对纯数值模型整体有效。DLinear 上三种文本相对 Numeric 的平均 MSE 改善约 7.9%–8.0%；PatchTST 上约 4.2%–4.3%。这说明多模态分支在结构层面确实能改善预测，尤其是当数值 backbone 本身较弱时。

问题二：怎样的文本模态有效？

当前结果显示 random_text、LLM-generated 和 external 三者较为接近，无法强证明 LLM-generated 和 external 文本的补充语义对预测有效，但依然由前面的结果数据可以看出，尽管在差距微弱的情况下，依然可以明显看出 LLM-generated 和 external 的效果普遍略强与 random 文本，说明模型有利用文本补充语义的能力。

问题三：如何使文本模态有效？

融合机制会影响文本模态融合的效果，且一个精心设计的融合机制可以对预测效果带来明显的提升。非线性融合提供稳定小幅收益（平均 0.6%–1.3%），频域融合在部分领域收益更大（最高达 17.3% 的 MSE 降低），但退化也更明显。

7.2 本项目完成清单

本项目不只是复现一个多模态模型，而是围绕"文本是否真的有用"做了一个较完整的控制实验：

1. 搭建了统一入口 `run.py` 和矩阵脚本，支持 `numeric`、`random_text`、

- `llm_generated`、`mm_tsflib`、`mm_tsflib_nonlinear`、`mm_tsflib_vot_freq` 等实验类型，保证所有对比实验在同一套代码和训练协议下运行。
- 在 8 个 Time-MMD 领域、2 个 backbone、多种 `pred_len` 下运行同口径实验，整理出 696 个主矩阵 run，全部通过完整性检查。
 - 构造了随机文本控制组，用字符级噪声替代真实文本，避免把"加了一个文本分支"误解解释成"文本语义有效"。
 - 构造了 ECNU LLM 自生成文本，通过严格的 prompt 约束和 audit 字段控制未来泄露，并缓存了所有生成过程的元数据。
 - 在 MM-TSFLib 基础融合之外，实现了非线性 gate 和频域分解融合，两种改进都采用了"保守初始化"策略，避免一上来破坏数值 backbone 的预测，将实验控制在仅探究融合机制影响层面。

7.3 局限性

局限性	影响
单 seed	当前结论只能写成趋势观察，不能声称多 seed 稳定
文本编码较弱	<code>use_fullmodel=0</code> 使用 input embedding + MLP + pooling，可能无法充分区分真实文本语义
没有尝试多个文本分支权重影响	由于时间和算力的限制，本项目仅测试了MM-TSFLib融合机制下文本分支 <code>prompt_weight=0.1</code> 的情况，并未改变和测试怎样的weight效果更明显，因此不同文本的效果差异不大，但依然可以看出LLM和extrnal文本效果有好于random文本的趋势

7.4 参考文献

- Zeng, A., Chen, M., Zhang, L., and Xu, Q. Are Transformers Effective for Time Series Forecasting? *AAAI*, 2023.
- Nie, Y., Nguyen, N. H., Sinthong, P., and Kalagnanam, J. A Time Series Is Worth 64 Words: Long-term Forecasting with Transformers. *ICLR*, 2023.
- Liu, H., et al. Time-MMD: Multi-Domain Multimodal Dataset for Time Series Analysis. *NeurIPS Datasets and Benchmarks*, 2024.
- Wang, S., Chen, P., Wang, Y., Qiu, W., Guo, C., Yang, B., and Shu, Y. Unlocking the Value of Text: Event-Driven Reasoning and Multi-Level Alignment for Time Series Forecasting. *ICLR*, 2026.